Introduction

Grid problems show up constantly in coding interviews because they look simple on the surface but hide a performance trap. LeetCode 1267, Count Servers that Communicate, is one of those problems. At first glance it feels like a counting exercise — scan the grid, find servers, check which ones have company. But the obvious way to check "does this server have company" is to compare it against every other server, and that's exactly the instinct interviewers are testing you on.

This problem teaches a pattern that reappears constantly in matrix and graph problems: when a relationship depends only on "shares a row" or "shares a column," you almost never need to compare elements pairwise. You can precompute aggregate counts along each dimension and then do O(1) lookups. That shift — from comparing objects to each other, to comparing objects against a precomputed summary — is one of the most useful tricks you can carry into other problems involving rows, columns, groups, or buckets.

It's also a great problem for practicing debugging discipline. A correct algorithm can still fail to run — and knowing how to separate "my logic is wrong" from "my code has a typo or missing import" is a skill in itself.

Problem Overview

You're given an `m x n` grid of 0s and 1s representing a data center floor plan. A 1 means a server sits in that cell; a 0 means the spot is empty. Two servers can communicate if they are in the same row or the same column — no matter how far apart they are in that row or column.

Your task: count how many servers can communicate with at least one other server. A server counts as "communicating" if there is at least one other server sharing its row, or at least one other server sharing its column (or both). A server that is completely alone — no other server in its row and no other server in its column — does not count.

Constraints: the grid can be as large as 250 rows by 250 columns, meaning up to 62,500 cells to examine. Every value is strictly 0 or 1, so there's no ambiguity in what counts as a server.

Example

Consider this 4x4 grid:

```
1 0 0 1
0 0 1 0
0 0 1 0
0 0 0 1
```

Row 0 has two servers (columns 0 and 3) — both communicate through their shared row. Column 2 has two servers (rows 1 and 2) — both communicate through their shared column. The server at row 3, column 3 is alone in row 3 (no other 1s) and alone in column 3 if you look only at rows 1–3 — wait, column 3 also has the server from row 0. Let's recheck: column 3 has servers at row 0 and row 3, so both of those communicate too.

Total communicating servers: 4 (row 0's two servers, column 2's two servers). Any server left over with no company in either its row or column — zero in this grid — is excluded.

The output for this grid is **4**.

Intuition

The first instinct most people have is: "to know if a server has company, I need to check every other server and see if it shares a row or column." That's a pairwise comparison, and pairwise comparisons scale badly — for `k` servers you're looking at roughly `k²` checks.

An experienced engineer pauses here and asks: what information do I actually need per server? Not a list of every other server — just two numbers: how many servers total sit in this server's row, and how many sit in this server's column. If either of those counts is greater than 1, the server has company. That's it. The identity of the other servers doesn't matter.

This reframes the problem from "compare pairs" to "count groups, then look up." Once you see that a row's total count or a column's total count is all you need, you realize you can compute both totals in a single pass over the grid, store them in two small arrays, and then do a second pass where every server's answer is an O(1) lookup.

The one trap here: don't add the row count and column count together and compare that sum to some threshold. A server sitting at the intersection of a busy row and a busy column would get counted using inflated logic. You need `row_count[r] > 1 OR col_count[c] > 1`, checked independently, not summed.

Brute Force Solution

Idea: For each server, scan its entire row and entire column looking for another server.

Algorithm: 1. Find all server positions. 2. For each server, scan across its row and down its column. 3. If any other 1 is found in either direction, count it.

Advantages: Simple to reason about, no extra data structures.

Disadvantages: For every server you potentially re-scan up to `m + n` cells. With up to `k` servers, this is roughly O(k × (m + n)) in the worst case, which degrades toward O(m² × n²) territory when the grid is dense. On a 250×250 grid full of servers, this gets slow fast.

Time complexity: O(k × (m + n)), worst case approaching O((m×n) × (m+n)) **Space complexity:** O(1) beyond storing server positions

Optimal Solution

Instead of asking "does this server have a neighbor," flip the question to "how many servers live in this row, and how many live in this column." Compute that once for every row and every column, then every server's answer becomes a lookup.

Step 1 — Tally pass: Create two arrays, `row_count` (size `m`) and `col_count` (size `n`), initialized to zero. Scan the grid once. Every time you find a 1 at `(r, c)`, increment `row_count[r]` and `col_count[c]`.

Step 2 — Decision pass: Scan the grid again. For every 1 at `(r, c)`, check if `row_count[r] > 1` or `col_count[c] > 1`. If either is true, add 1 to the total.

Row	Server Count	Column	Server Count
0	2	0	1
1	1	1	0
2	1	2	2
3	1	3	2

Row 0 clears the bar by itself (count 2). Rows 1 and 2 look lonely on their own, but their servers sit in column 2, which has count 2 — so they qualify through their column. Row 3's server sits in column 3, which also has count 2, so it qualifies too in this particular example (note this differs slightly from the simplified walkthrough grid used earlier — always trust the tally table over a quick mental scan).

This is the whole trick: pay the cost of counting once, then every server's status is a constant-time lookup.

Python Code

```
from typing import List


class Solution:
    def countServers(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])

        row_count = [0] * rows
        col_count = [0] * cols

        # First pass: tally servers per row and per column
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    row_count[r] += 1
                    col_count[c] += 1

        # Second pass: count servers that have company
        total = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1 and (row_count[r] > 1 or col_count[c] > 1):
                    total += 1

        return total
```

Code Walkthrough

- `from typing import List` — required because the method signature uses `List[List[int]]`. Skipping this raises a `NameError` at call time, even though the algorithm itself is correct.
- `row_count` and `col_count` — two arrays sized to the grid's dimensions, starting at zero. These hold the aggregate counts the whole solution depends on.
- First nested loop — visits every cell once. On finding a server, it bumps both the row's and column's tally. This is the "group counting" step.
- Second nested loop — visits every cell again. For each server, it checks the precomputed tallies instead of re-scanning neighbors. This is the O(1) lookup step that replaces pairwise comparison.
- The condition `row_count[r] > 1 or col_count[c] > 1` is evaluated independently for row and column — never summed — to avoid double-counting servers at busy intersections.

Dry Run

Grid:

```
1 0 0 1
0 0 1 0
0 0 1 0
0 0 0 1
```

Tally pass: - Row 0: servers at columns 0, 3 → `row_count[0] = 2` - Row 1: server at column 2 → `row_count[1] = 1` - Row 2: server at column 2 → `row_count[2] = 1` - Row 3: server at column 3 → `row_count[3] = 1` - `col_count = [1, 0, 2, 2]`

Decision pass: - (0,0): row_count[0]=2 → counts - (0,3): row_count[0]=2 → counts - (1,2): col_count[2]=2 → counts - (2,2): col_count[2]=2 → counts - (3,3): col_count[3]=2 → counts

Total = 5 for this particular grid layout (differs from the narrower 4-count example because column 3 here has two servers; always re-derive from your actual grid rather than assuming).

Complexity Analysis

Time: O(m × n) — two full passes over the grid, each visiting every cell exactly once. No pass depends on how many servers exist, so it scales cleanly even on a dense 250×250 grid.

Space: O(m + n) — only the two tally arrays are extra storage, sized to the number of rows plus the number of columns, independent of grid density.

This is correct because every piece of information the second pass needs (row and column totals) is fully computed before that pass begins, and no cell is ever revisited more than the fixed two times.

Alternative Solutions

A single-pass variant is possible using `collections.Counter` or plain dictionaries keyed by row and column index, built while first scanning for server positions, then checked in one combined loop over just the stored positions instead of the whole grid. This can save a bit of time on sparse grids (few servers) since the second pass only touches actual server coordinates rather than every cell. For dense grids, the difference is negligible — you're still bounded by O(m × n) to build the tallies in the first place.

Edge Cases

- **Empty grid or no servers:** every tally stays zero, result is 0.
- **Exactly one server total:** it's alone in both its row and column, result is 0.
- **Every row and column has exactly one server (diagonal pattern):** no row or column count exceeds 1 anywhere, result is 0.
- **Entire grid filled with servers:** every row and column count equals the grid's full dimension, so every server counts.
- **Single row or single column grid:** behaves correctly since the "other" dimension's count is naturally capped at 1 if only one cell per column exists, but the shared dimension can still exceed 1.

Common Mistakes

1. Forgetting `from typing import List`, causing a `NameError` despite correct logic.
2. Summing row count and column count instead of checking them independently, which miscounts servers at busy intersections.
3. Checking neighbors pairwise instead of precomputing tallies, leading to slow, nested-loop-heavy solutions that time out on large grids.
4. Off-by-one errors when initializing tally array sizes (using `len(grid)` for columns instead of `len(grid[0])`).
5. Only running the tally pass and forgetting the second decision pass, returning an incorrect or incomplete count.

Interview Questions

- How would you solve this if you needed to return the actual list of communicating server coordinates, not just the count?
- What changes if "communicate" required servers to be adjacent in the same row/column, not just sharing it?
- Could you solve this with a single pass instead of two? What would you need to track differently?
- How would your approach change if the grid were extremely sparse (few servers, huge grid)?

Similar Problems

- **LeetCode 1041 — Robot Bounded in Circle:** another grid-adjacent problem relying on tracking aggregate state rather than pairwise simulation.
- **LeetCode 36 — Valid Sudoku:** uses the same row/column (plus box) tally pattern to validate constraints in one pass.

- **LeetCode 304 — Range Sum Query 2D:** builds on precomputed row/column-style aggregates (prefix sums) to avoid recomputation.
- **LeetCode 1162 — As Far from Land as Possible:** grid problem where precomputing distances beats pairwise comparisons.

Key Takeaways

When a relationship in a grid depends only on sharing a row or column, don't compare elements to each other — count each row and column once, then look up the answer. This turns a potentially quadratic (or worse) pairwise comparison into two clean linear passes: O(m × n) time, O(m + n) space. It's a pattern worth recognizing anytime "shares a group" appears in a problem statement.

Watch the Video

For the full walkthrough — including the real debugging moment where a missing import caused a `NameError` — watch the video here: {LINK}

About the Series

This solution is part of *Fun with Learning Technology*, a series that breaks down LeetCode problems, real bugs, and the engineering intuition behind them — not just the final code, but how an engineer actually gets there, mistakes included.

Call To Action

If this breakdown helped, like the video on YouTube and subscribe for more daily LeetCode walkthroughs. For deeper write-ups like this one, subscribe to the Substack newsletter. Drop a comment with your own approach or questions, and share this with anyone prepping for interviews.

Quick Review

Pattern + trigger: grid where a row/col has >1 server?

Row/column tally pattern — when a cell's row count or col count exceeds 1, it 'talks'. Trigger: grid connectivity without BFS/DFS.

Time complexity for counting communicating servers?

O(m*n) — one pass to tally rows/cols, one pass to check each server.

Space complexity for the row/col tally approach?

O(m+n) for row-count and col-count arrays, not O(m*n).

Trickiest bug in this problem?

Checking rowCount/colCount inline while still tallying — must finish counting pass before the checking pass, or counts are incomplete.

Why not BFS/DFS union-find here?

Overkill: 'same row or same col' already defines connectivity directly, no graph traversal needed to find components.

Interview follow-up: server with 0 others in row AND col?

It's isolated — excluded from the count; only servers whose row count >1 or col count >1 are counted.

Count Servers that Communicate — free Python cheat sheet